# Playing Card Recognition Final Report

## Mission and Introduction

The idea of the project comes from the fact that card games are often played by **elders**. As it's known, as years go by, new **problems** come up, and they may be related to **eyesight**. Hence, we tried to develop a computer vision algorithm that aims to solve this problem for our beloved grandparents. Since "Briscola" is a very well-known card game, we started by **recognizing** those **cards**, even though the project is extendible to other card sets.

The final objective achieved is a live card recognition algorithm, which exploits all the classical computer vision techniques such as image segmentation, blob detection, keypoint description and matching. The code works over a controlled environment, in which there is no reflecting light on cards and there is a plain colored towel (preferably green) on the table.

To get this result, this path has been followed:

- **Deck images gathering**
- **Deck creation** as an array of card objects, with keypoints and descriptors
- **Main algorithm structure**: compute background parameters, segmentation, blob analysis, keypoints detection and feature computation, matching and displaying
- **File versions**:
    1. **Image** recognition file: This file was created to tune the algorithm by firstly recognizing cards in static images, which contained single cards, overlapped cards and object disturbances. The parameter tuning was achieved by keeping track of parameters and results obtained on a benchmark set of different images. Finally, they were saved on a spreadsheet file from which we have chosen the ones performing better
    2. **Live** file: This file was the prosecution of the first, in which the attention is shifted from static images to sequences of live frames obtained through a camera
    3. **Video** file: a program which recognizes cards from videos, to let clients try the algorithm without having to set up the environment.

Through this journey the algorithm was shaped in relation to the problems faced, trying to reach a fast and robust solution to the problem, but only in controlled environments. We will go deeply into each part of the process.

## Photos and deck creation



This part led to the creation of an object array called deck, which contains the card image, card ID, card keypoints and the descriptors associated to each one of those.

The first step was to **take card pictures in a carefully controlled environment**. The second step was to **apply the algorithm** that will be explained in the next point and to store the results, as well as the image, into an object, called Card, with the afore mentioned attributes. The third step is to **create an array of**

**Card objects** containing the whole deck. This part of the code is executed as a sort of initialization process, so it is **offline** (not in real-time), and it happens before starting the card recognition process.

In a more abstract view, deck creation is a black box in which you input deck images (not necessarily briscola ones) and it outputs the Card object array.

Next is a photo of the setup we arranged. As you can see, the camera is fixed pointing towards the green towel which is the background.



# Main functions

- **Compute background attributes**: Takes as input a background picture, and computes the mean Intensity as well as the inverted covariance matrix, which are the two parameters used to calculate Mahalanobis distance
- **Compute Mahalanobis distance**: Takes as inputs the afore mentioned parameters and an image and calculates the Mahalanobis distance, returning the distance for each pixel
- **Segment image**: Takes as inputs the image, the Mahalanobis distances array, the threshold for Mahalanobis distances, and a kernel for image dilation. In substance, the function segments the image relying on the threshold given as input. After that, since some green pixels of the image may be lost, the dilation helps to close in-foreground-region gaps. At the

end a mask is applied, which has value one for the foreground segmented objects and zero for the background, to prevent the algorithm to find keypoints in the background.

- **Detect blobs**: Takes as inputs the image and the connectivity type and returns an array of blobs
- **Compute features and descriptors**: Takes as input the cropped blob portion of the image as well as the maximum number of keypoints and the detector. It returns a tuple with keypoints and descriptors array. Here it is all done by a command that detects keypoints and describes them with the selected descriptor (Sift in this case).
- **Matching**: the matching does not have a proper function since it is only a line of code. We use Brute Force knnmatcher integrated command, extracting the two best matches for each comparison between a blob descriptor and each deck card descriptor.

# Algorithm description

The previously described functions are all deployed in real time in a while loop as they have been presented. The core of the algorithm is composed by two heuristic specific for this problem.

- Main heuristic

Instead of just matching, we decided, for robust purposes, to match, apply a Lowe ratio on the two best matches, and then re-apply the matching process and Lowe ratio filtering but in a reverse manner (from deck card to blob) to keep only biunivocal correspondences. In addition to that, the matches that passed both rounds are filtered by RANSAC, which detects outliers by establishing a homography between matching keypoints.

Lastly a 1D vote is performed for every card in the deck. Note that there could be multiple peeks in voting for the same blob since cards could be overlapped

To establish a guess two thresholds have been defined:

- **Confidence threshold**: ratio between the absolute frequency of matches for a certain deck card and the absolute frequency of matches for the most matched card in deck.
- **Absolute threshold**: the absolute frequency of matches for a card must be higher than a certain absolute frequency threshold

If both absolute and relative number of matches for a card are higher than these thresholds then the card is considered as a guess.

In this way we addressed the problem of displaying multiple cards guess for a single blob of overlapped cards.

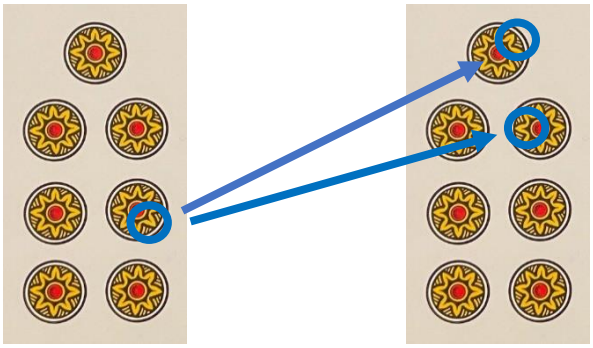
The problem with this algorithm was related to cards with highly repetitive card patterns, and we were not able to understand why, since it worked perfectly for other cards. We reached the conclusion that the algorithm rejected matches of repetitive features because of two main things:

- Lowe ratio filtered them out as they appeared in too many images equally, and so the two best matches distances were comparable.

- Homography with Ransac or in general geometrical based voting works against this specific case. Indeed, a repetitive pattern could be matched all over the card and geometric position is not respected between good matches.

Since we could not exclude these two things since they were working greatly with other cards, we opted for a specific heuristic.

Here is an example of what we think is happening with repetitive patterns.



## ➢ Auxiliary heuristic

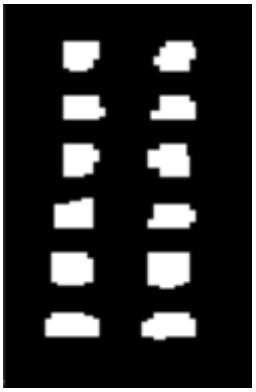
This heuristic is based on the fact that repetitive patterns can be recognized easily and it's simple to spot them.

What the algorithm does could be divided into two sections:

- Guess what the right seed is. The guess is only between coins and cups, since with the others the program didn't have any issue
- Count the number of blobs in a segmented card

To achieve the first result, a 1D voting on all deck cards is done in the main heuristic, after the first Lowe filtering. Then another absolute threshold is applied ("abs_thresh2") which filters out not so robust results in voting. Then we compute the maximum between the sum of matches for every seed. In this way we obtain a maximum likely seed guess.

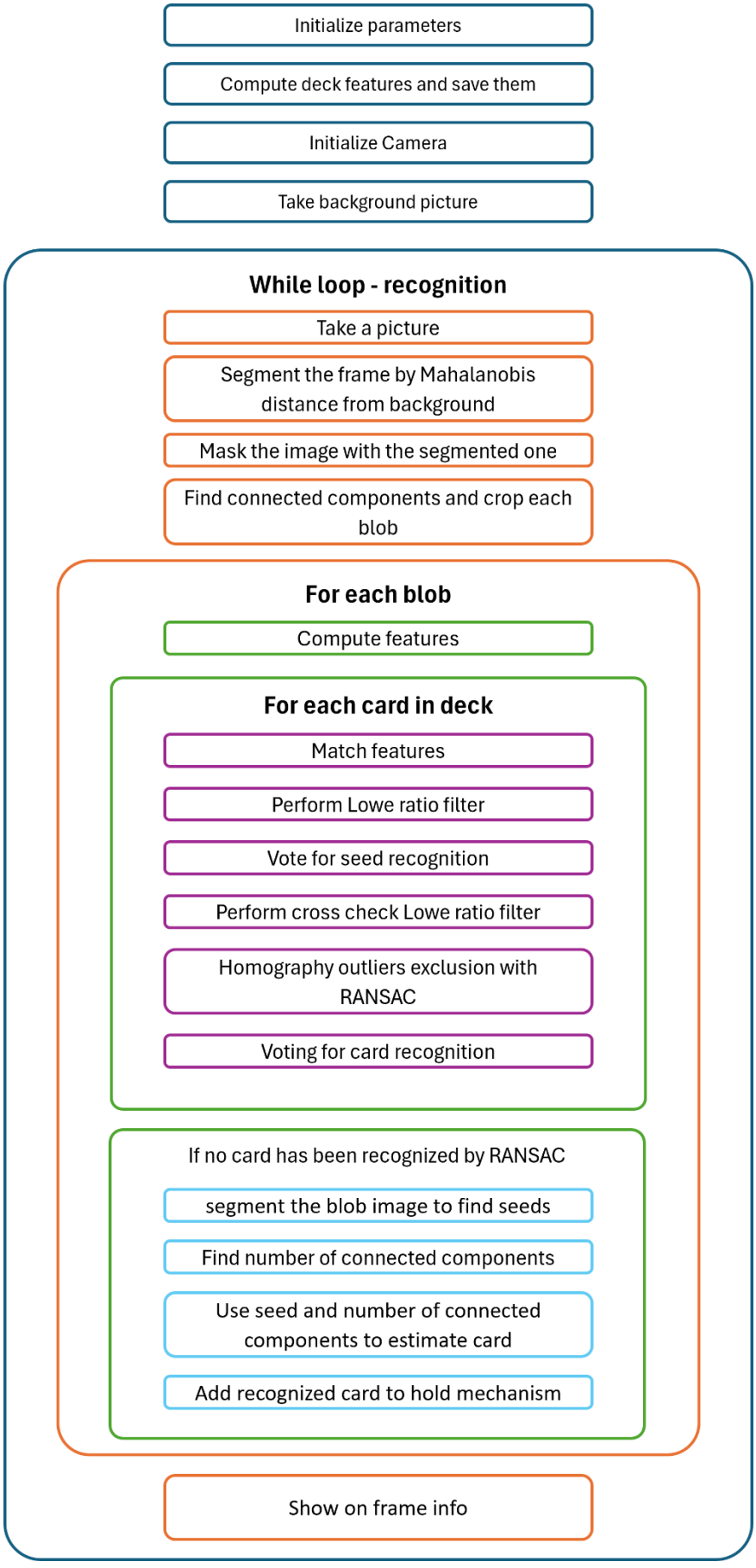
After this, we segment the card itself to obtain only the figures on it. Here is an example with 6 of cups. We couldn't achieve robustly a segmentation of an entire cup, thus we decided to consider as a cup two connected components.



Finally, if the main algorithm has not found anything, the auxiliary one would get a result.

It's important to point out that this new algorithm is not as robust as the main one and should be improved.

# Algorithm flowchart



# Algorithm versions

During the development of the algorithm many programs were written to test the recognition program in different ways: test images, Real time stream, test videos

- **Test images recognition**

The simplest thing to do was to first recognize cards in images. On top of that, we built an architecture to study the results, that would run the algorithm on a test set of images and in that way we could correlate each run tuned with different parameters.

We prepared a set of test images, located in the folder “test_images”, that are of this kind:



The results we would get from every test are:

- <card_name> found with a certain confidence
- <card_name> not found
- Elapsed time

```
Starting Test14
2_SWORDS, Confidence: 100% OK
5_SWORDS, Confidence: 76% OK
7_BATONS, Confidence: 38% OK
1_SWORDS not found
Elapsed time: 7.8540 seconds
```

Then we could also display averages for the whole test set.

```
Total cards found wrt total images 28\29
Right cards found wrt total images 28\29
Cards not found wrt total images 1\29
```

```
Elapsed average time: 3.336
Average frequency: 0.3
Average right confidency 69.93
Average wrong confidency 0
Min right confidency 11
Max wrong confidency 0
Data has been saved to results.xlsx
```

As it can be seen, in this way we could have both a local view of every test and a general view of how the algorithm performed.

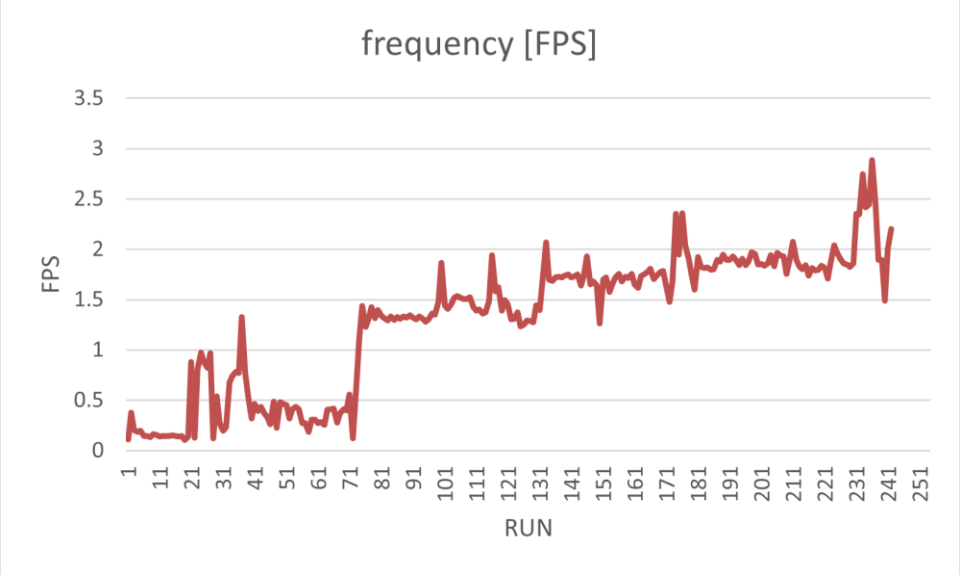
An important parameter to consider is the average time the algorithm takes in general. It depends, of course, on specific hardware resources, but to use the algorithm also in real time we needed it be fast enough. Notice also that speed depends highly also on the number of cards in the scene.

Other important parameters are the ones concerning confidence. Indeed, we were looking for a robust algorithm to deploy.

A big mistake we made in the early phases was not making test set big enough. Indeed, we were not aware of the fact that the algorithm struggled with highly repetitive patterns in cards. That's why in the benchmark program only the main heuristic is used and not also the auxiliary one.

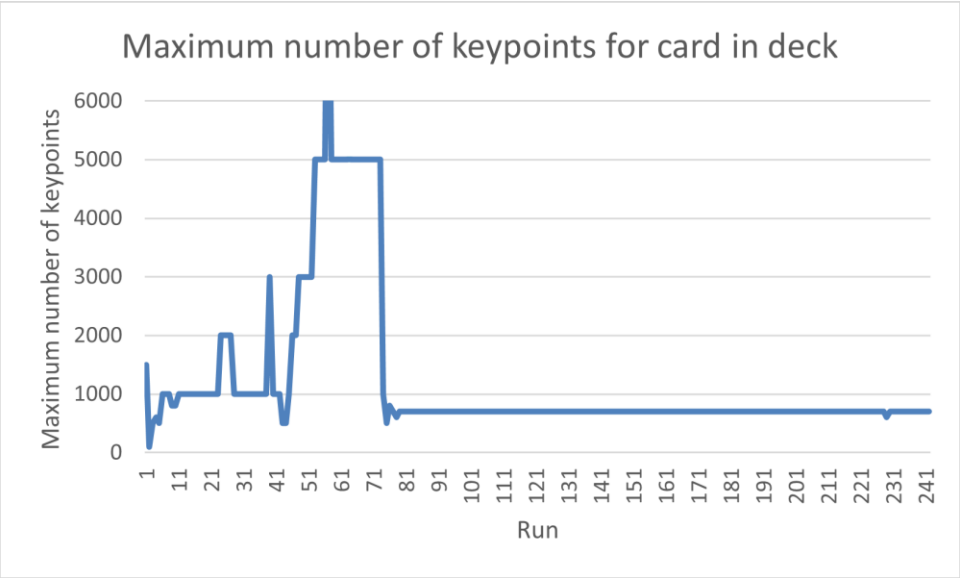
Here are some graphs of how the tuning process went:

- **Algorithm speed**

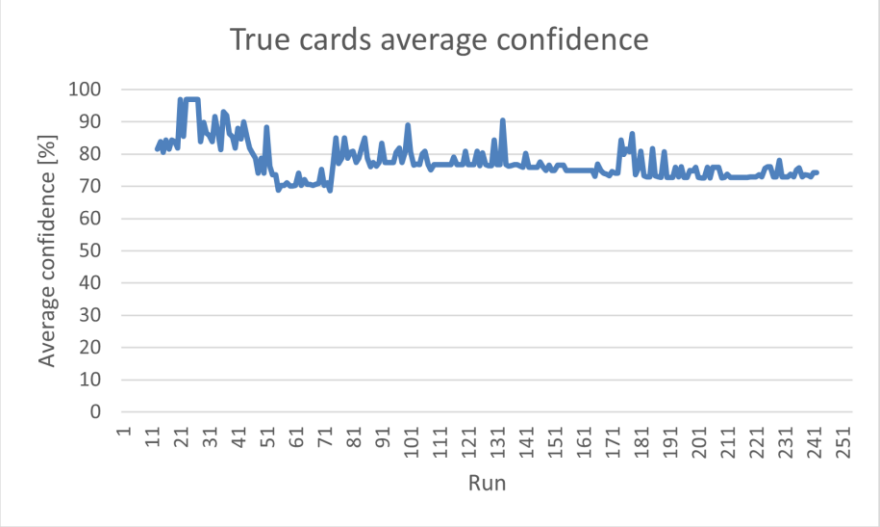



As it can be seen, we started from a very low FPS rate. That's because our algorithm was highly inefficient because we used FLANN matcher with many keypoints. Also, it could appear strange at first glance but computing Mahalanobis distance in real time is highly inefficient and we tried as much as we could to make it more efficient.

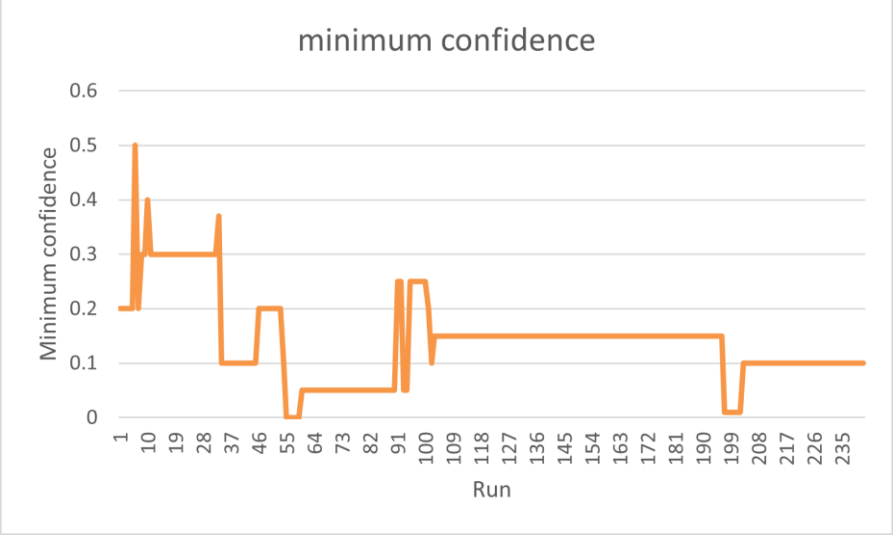
The big spike around run 70 was due to a descrease in the number of maximum deck keypoints as can be seen in next figure:




- **Confidence**



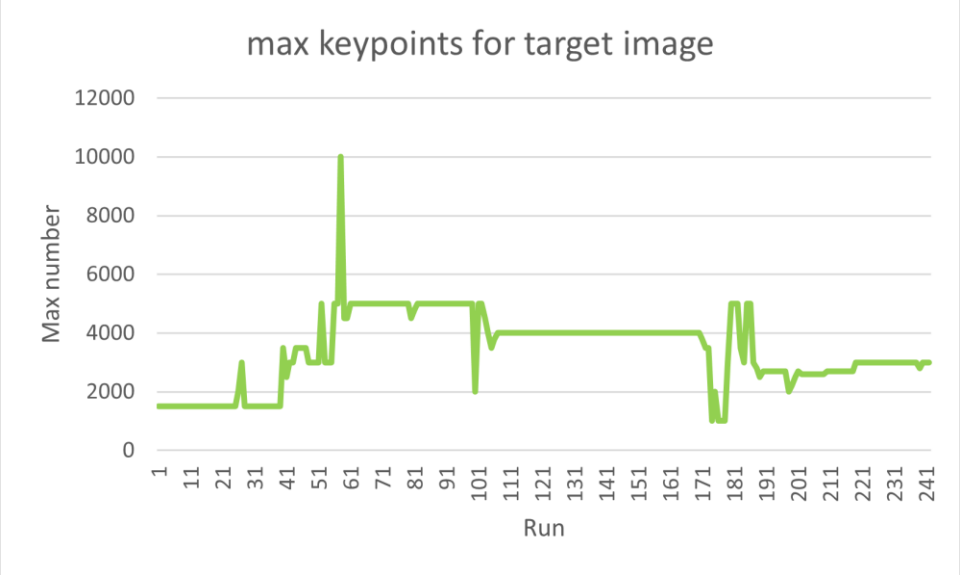

Speaking of the average confidence, it did decrease a bit but then stabilized. This is because overtime more and more cards with lower confidence were recognized.



Indeed, here is the demonstration of why average confidence was decreasing. That is because we lower down the confidence threshold to accept more guesses from the algorithm.

- **Number of keypoints used for target image**

For what concerns number of key points used, initially we increased exponentially the number of it and the tried to return to a normal value.




## ➢ Real time program

Once we tuned our first algorithm on a small test set, we decided to move to real time program.

In here we found out that the program had issues with highly repetitive patterns.

We tried many things with main heuristic (1D/2D/4D voting and RANSAC) but everything failed. Thus, we had to develop a new heuristic specific for the card that failed to be recognized.

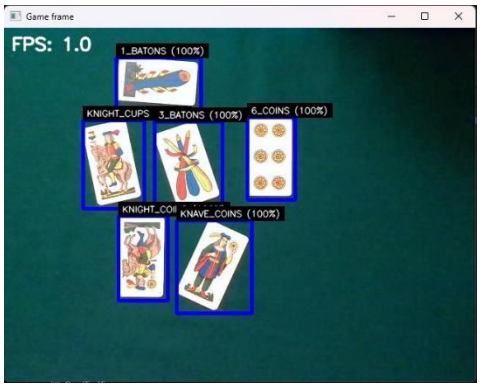
A feature that we developed for this kind of task was automatic background acquisition for each initialization of the program. That's to make sure the recognition would be more robust with setup changes.

## ➢ Video stream program

Finally, we developed this final version that works on saved video streams. That was for the sake of this project since it's hard to get a fully working setup without major tuning. In the code a video can be chosen. We proposed four of them to test the algorithm out.

Looking at it in perspective it would probably have been better if we considered first this kind of process instead of the real time program.

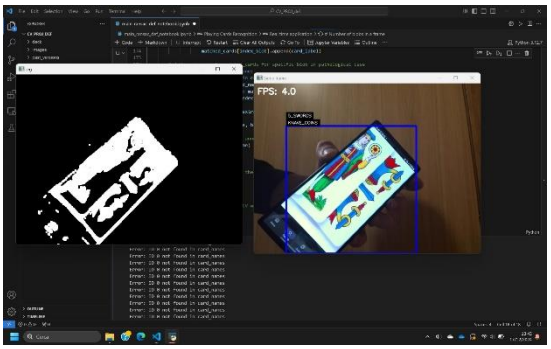
# Results




In the end, as one may notice by looking at videos in the "**video_results**" folder, results are great! The main algorithm, combined with the specific heuristics deployed for highly repetitive card patterns, **recognizes all 38 cards**, both **singularly and overlapped**, **avoiding outliers**: all this with a **discretely fast execution time**, fast enough to be **deployed live**. The algorithm has **high robustness** in highly **controlled environments**. However, it is not always possible to have such care for the setting, hence there may be **situations in which it is frail** and might require fine tuning of background parameters as well as denoising for light flickering phenomena. Furthermore, the specific heuristic **doesn't work when the "pathological" cards are overlapped**, but that's a minor problem.

# Next steps

The aim is to improve the project and expand the algorithm to other decks. In fact, future updates will be regarding:




- **Robustness to environment**: we are planning to address the problem of background setup by developing an automatic, adaptive threshold selection and filtering for noise reduction
- **Overlapping and occlusion handling in specific Heuristic**: the idea is to handle the recognition of cards with repetitive patterns also when overlapped and occluded
- **Extension to different decks**: the concept is to introduce different folders with different deck images and, once you select the deck, it automatically creates the deck array and gets ready to play a game with the deck selected

# How to use

1. **Dependencies**:

Ensure all required libraries are installed:

- OpenCV
- Matplotlib
- Numpy

2. Read the notebook "**main_notebook.ipynb**" for better documentation on the python algorithm

3. How to use “**main_real_time.py**”
   3.1. Create a good setup with possibly green towel and camera that frames the table from top to bottom
   3.2. Select the right video capture source
   3.3. Run the algorithm
   3.4. Iteratively tune the parameters in section 4 to optimize the result

4. How to use “**main_stream_video.py**”
   4.1. Load the video from "test_videos/" in section 4
   4.2. Choose the output path with name e.g. "video_results/out.mp4"
   4.3. Run the algorithm
   4.4. Iteratively tune the parameters in section 4 to optimize the result
5. How to use “**main_images_benchmark.py**”
   5.1. Run the algorithm on the test set
   5.2. Iteratively tune the parameters in section 4 to optimize the result

## Use limitation

1. **Setup**
   1.1. The setup must be sufficiently bright, avoiding light reflection on card surface
   1.2. It is recommended for the background to be mostly uniform and plain, green towel is the best candidate
   1.3. Distance from camera to table shouldn't be greater than 50cm
   1.4. For all the run, camera must point exclusively to background
2. **Parameters**
   2.1. segment_threshold: 20-60, for brighter background choose lower threshold
   2.2. deck keypoints: 1000-2000
   2.3. blob keypoints: 2000-5000, the farther the camera, the higher the number of keypoints.
   2.4. Lowe thresholds: 0.6-0.9, the lower, the stricter
   2.5. confidence: 0.2-0.5, 0.3 used
   2.6. abs_thresh1: 5-15, the higher the stricter. Recommended 8
   2.7. abs_thresh2: 10-20, the higher the stricter.
   2.8. blobs_min_height/max_height/min_width/max_width: are fixed values. Should be optimized when changing SETUP
3. **Important disclaimer**
   3.1. deck cards are not 40 but 38, since two were missing (6 swords, king batons)
   3.2. parameters are not to be thought of as independent parameters. Indeed, they are interconnected, tuning is required when nudging or when making changes to the setup (i.e. the recognizer performs differently at different hours because of lighting conditions)
   3.3. When frame capture is active, wait a few seconds for the background to be captured
   3.4. the algorithm implemented in "main_images_benchmark" do not use the specific heuristic for repetitive patterns, since in initial test images there were not repetitive patterns.